

LC 1263: Minimum Moves to Move a Box

© Authored by: **Arpit Raj** | LNMIIT Jaipur | July 27, 2026

Algorithm Notes & Strategy

Outer BFS

- Only care about `box row` , `box col` and every level of BFS is **one box push**.
- **Outer queue:** `{boxrow, boxcol, playerrow, playercol}`
- We still store player's position because after pushing, player goes into the old box position.

Inner BFS

- Before pushing, we must check if person can reach pushing position *[without crossing walls / box]*.
- **Time Complexity:** `TC =  $O((nm)^2)$`

Mechanics & Example

Current state: `(box r, box c, player r, player c)`

TRY 4 push directions:

Suppose **right**, then:

`newbox → box → right`

Player must stand to the **left** of the box to make this possible:

- `need r = box r`
- `need c = box c - 1`

Now call a function:

`canReach(player r, player c, need r, need c, box r, box c)`

- If **false** → continue
- If **true** → perform push and position change

C++ Implementation

```
class Solution {
public:
    int n;
    int m;
    vector<pair<int,int>> dir={{-1,0},{0,-1},{0,1},{1,0}};
    bool canreach(int needpr,int needpc,int cbr,int cbc,int cpr,int
cpc,vector<vector<char>>& grid){

        queue<pair<int,int>> q;
        vector<vector<int>> seen(n,vector<int>(m,0));

        q.push({cpr,cpc});
        seen[cpr][cpc]=1;

        while(!q.empty()){
            int r=q.front().first;
            int c=q.front().second;

            q.pop();
            if(r==needpr && c==needpc){
                return true;
            }

            for(auto &it:dir){
                int nr=r+it.first;
                int nc=c+it.second;

                if(nr>=0 && nc>=0 && nr<n && nc<m && !seen[nr][nc]){
                    if(grid[nr][nc]!='#' && !(nr==cbr && nc==cbc)){
                        //cant pass through wall or box

                        seen[nr][nc]=1;
                        q.push({nr,nc});
                    }
                }
            }
        }

        return false;
    }
};

int minPushBox(vector<vector<char>>& grid) {
    n=grid.size();
```

```

m=grid[0].size();

int br;
int bc;
int pr;
int pc;
int tr;
int tc;

for(int i=0;i<n;i++){
    for(int j=0;j<m;j++){

        if(grid[i][j]=='B'){
            br=i;
            bc=j;
        }

        if(grid[i][j]=='S'){
            pr=i;
            pc=j;
        }

        if(grid[i][j]=='T'){
            tr=i;
            tc=j;
        }
    }
}

//a queue to store {{br,bc},{pr,pc}}

queue<pair<pair<int,int>,pair<int,int>>> q;

//a seen matrix using constraint
int seen[20][20][20][20]={0};

q.push({{br,bc},{pr,pc}});
seen[br][bc][pr][pc]=1;

int ans=0;
while(!q.empty()){

    int currsz=q.size();

    while(currsz--){
        int cbr=q.front().first.first;
        int cbc=q.front().first.second;

```

```

        int cpr=q.front().second.first;
        int cpc=q.front().second.second;

        q.pop();

        if(cbr==tr && cbc==tc){
            return ans;
        }

        for(auto &it:dir){ //trying push in every direction
            int nbr=cbr+it.first;
            int nbc=cbc+it.second;

            int needpr=cbr-it.first;
            int needpc=cbc-it.second; //valid player r and c to push

            if(nbr>=0 && nbc>=0 && needpr>=0 && needpc>=0 && nbr<n &&
            needpr<n && nbc<m && needpc<m){
                if(grid[nbr][nbc]!='#' && grid[needpr][needpc]!='#' && !
                seen[nbr][nbc][cbr][cbc]){
                    if(canreach(needpr,needpc,cbr,cbc,cpr,cpc,grid)){
                        //cant magically teleport , isliye ye function
                        seen[nbr][nbc][cbr][cbc]=1; //player goes to where
                        the box was kept before push
                        q.push({{nbr,nbc},{cbr,cbc}});
                    }
                }
            }
        }

        ans++; //ek bfs level matlab ek push

    }

    return -1;
}
};

```